Blockchain-Based Library Book Lending Tracker

Technical Report, Individual Assignment 2 (Formative 2)

Name: Golden Sash Munyankindi

Date: October 2026

GitHub repository: https://github.com/goldensash1/Assignment-2--F-A-Extending-the-Blockchain-Based-Library-Book-Lending-Tracker

Demo video: https://youtu.be/A9BjZngE3Ks

1. Introduction

In Formative 1 I built a command line program in C. It saves every borrow, return and overdue event as a signed block in a chain of SHA-256 hashes. In that version, a block went onto the chain as soon as the librarian saved the event.

In this assignment I changed the same program in these ways:

- When a book is returned, the program now creates a token reward transaction by itself. The reward is 10 coins for an on-time return and 5 coins for a late return. If a book is overdue and not returned, no transaction is created.
- New blocks do not go onto the chain straight away. They wait in a pending pool. A miner has to mine them with proof of work first. Only after that is the record final and the member's balance changed.
- Member balances can be kept in two ways: the UTXO model or the account-based model. The user picks one when the program starts, and it is used for the whole session.
- Mining can be done alone (solo), in a pool, or with rented cloud mining power.

The assignment brief sometimes says "attendance" and "absent student". In my library system the same ideas are a lending event (borrow, return or overdue) and a book that is overdue and not returned. I use the library words in this report.

2. What Changed from Formative 1

The code now has 16 source files. I kept the Formative 1 files for the registry, login, crypto and input, and I changed blockchain.c and main.c. These are the new files:

- **transaction.c:** the transaction struct, its SHA-256 ID, and the coin amounts.
- **utxo.c:** the UTXO set, choosing coins, change, fees and the double-spend checks.
- **account.c:** accounts with a balance and a nonce, and a linked-list history for each account.
- **ledger.c:** the list of confirmed transactions. It sends each one to the model that was chosen.

- **pending.c:** the pending pool of blocks that are not mined yet.
- **mining.c:** solo, pool and cloud mining, the difficulty setting, and a benchmark.
- **lending.c, tokens.c, app.c:** the menu actions and the shared program data.
- **storage.c:** saving and loading the binary files for the chain, the pool and the ledger.

2.1 Block structure

All the Formative 1 fields are the same. I added the two fields the brief asks for. I also added three fields that proof of work needs.

Table 1: Block fields

Field	Type	Purpose
index, timestamp, book_id, book_title, member_id, member_name, action, recorded_by, previous_hash, signature, sig_len, hash	same as Formative 1	No change
token_reward	int	NEW. Coins earned: 10 for an on-time return, 5 for a late return, 0 for anything else
tx_id	char[65]	NEW. SHA-256 hash of the reward (or transfer) transaction. It is 64 zeros if the block has no transaction
difficulty	int	How many leading zeros this block was mined with (1 to 4)
mined_by	char[40]	Who mined the block: a username, POOL, or CLOUD:username
nonce	unsigned long long	The number the miner changes until the hash is good enough

Each block keeps its own difficulty. If the difficulty is changed later, old blocks can still be checked against the difficulty they were mined with. The action field can now also be TRANSFER, which is a manual token transfer between two members.

3. How Lending Events Create Token Transactions

Every lending action makes a new block. The action decides if a transaction is made:

Table 2: Lending events and transactions

Event	Token reward	Transaction created
BORROWED	0	None
OVERDUE (book not returned)	0	None. The block shows the book is

		not back, but no coins move
RETURNED on time	10	REWARD: LIBRARY pays 10 coins. The member gets 9 and the miner gets a 1-coin fee
RETURNED late	5	REWARD: LIBRARY pays 5 coins. The member gets 4 and the miner gets a 1-coin fee
TRANSFER (typed in by the user)	0	TRANSFER: from one member to another. The sender also pays the 1-coin fee

A return is late if more than 14 days passed since the borrow, or if a librarian already marked the loan as OVERDUE. I added the second rule so that I can show a late return without waiting 14 days.

When a book is returned, the function lending_return() in lending.c does these steps:

1. It finds the open loan. It looks at the chain first and then at the pending pool, as one list. So a book borrowed in a pending block already counts as on loan.
2. It decides if the return is on time or late. The reward is 10 or 5.
3. It builds the reward transaction. The sender is LIBRARY and the receiver is the member. The amount is the reward minus 1, and the fee is 1. The transaction ID is the SHA-256 hash of these fields.
4. It asks the ledger to accept the transaction. The UTXO model picks and locks some of the library's coins. The account model checks the library's nonce and balance.
5. It makes the RETURNED block with token_reward and tx_id filled in. It signs the block with the librarian's ECDSA private key and puts it in the pending pool together with the transaction.

The reward and the transaction ID are part of what the librarian signs. So if anyone changes one of them later, the signature or the link stops matching.

Where the coins come from: when the genesis block is made, a coinbase transaction gives the LIBRARY account 10,000 coins. The tx_id of the genesis block points to that transaction. Members start with 0 coins. They can only get coins from confirmed returns, or from a transfer from another member.

4. The Pending Pool and How Mining Confirms Blocks

The pending pool (pending.c) is a queue. The first block in is the first block mined. Each entry holds a signed block and, for returns and transfers, its transaction. The pool is saved to pending.dat after every change, so it is still there after a restart. A pending block has no index, previous_hash or nonce yet, because its place in the chain is only decided when it is mined.

While a block is pending, its transaction has passed the checks and the sender's funds are reserved. But no balance has changed yet. Mining is the step that confirms the block. For each block, oldest first, the miner does this:

6. It checks the block. The librarian's signature must be valid, the transaction must hash to the block's tx_id, and the amount must match token_reward. A bad block is dropped and its reserved funds are released.
7. It puts the block in its place. It sets the index to the chain length and previous_hash to the hash of the last block. It also saves the difficulty and the miner's name.
8. It does the proof of work. It starts at nonce 0 and adds 1 each time, hashing the block again, until the SHA-256 hash starts with enough zeros.
9. It adds the block to the chain and confirms the transaction in the ledger. If the ledger refuses the transaction, the block is taken off again, so the chain and the ledger always agree.
10. It saves the chain, the ledger and the pool. It prints the UTXO set (UTXO model) or the updated accounts (account model).

After the pool is mined, the miner is paid with a new coinbase transaction. It pays 50 coins for each confirmed block plus the fees of the transactions in those blocks.

When the program starts again, the pending transactions are checked again under the model chosen for that session. Any that are no longer valid are dropped and a warning is shown.

5. Transaction Models

Both models use the same transaction struct. It holds the type, sender, receiver, amount, fee, nonce, time and a short note. The transaction ID is the SHA-256 hash of these fields. Amounts are stored as whole numbers of hundredths of a coin. This way fees and pool shares add up exactly, with no rounding errors from decimal numbers.

Every confirmed transaction is added, in order, to a list in ledger.c, which is saved as ledger.dat. When the program starts, this list is replayed into the chosen model. Both models are built from the same list, so switching between them does not change any balance. Only the way balances are stored and checked is different. Figures 8 and 13 show the same balances under each model.

5.1 UTXO model (utxo.c)

The UTXO set is a dynamic array. Each item has a txid, a vout, an owner, an amount and two status fields. A balance is not stored anywhere. It is the total of the outputs a member owns.

- **Inputs and outputs:** a transaction spends whole old outputs as its inputs. The program picks the sender's oldest free outputs until they cover the amount plus the fee. Output 0 pays the receiver.
- **Fee:** the fee is not an output. It is the inputs minus the outputs. The miner gets it in the coinbase transaction. Every reward and transfer has a fixed fee of 1 coin.

- **Change:** if the inputs are worth more than the amount plus the fee, output 1 sends the extra back to the sender. Example: John spends his 9-coin output to send 3 coins. 3 go to the receiver, 5 come back to John as change, and 1 is the fee (Figure 10).
- **Not enough inputs:** if the sender's free outputs cannot cover the amount plus the fee, the transaction is rejected.
- **Double spending:** when a transaction enters the pending pool, its inputs are marked with its ID (reserved_by), so no other pending transaction can use them. When the transaction is confirmed, the inputs are removed from the set. A transaction is rejected if an input is missing (already spent), is reserved by another transaction, belongs to someone else, or is listed twice (Figure 12).
- **Check at confirmation:** all inputs are checked again and they must all be confirmed. The set is changed only after every check passes.
- **Display:** the full UTXO set, with the owner and status of each output, is printed after every confirmed block.

One problem I had to fix: the library starts with one single 10,000-coin output. After the first return, that output was locked by the pending reward, so a second return was refused until the first one was mined. Bitcoin lets a sender spend the change from its own unconfirmed transaction, so I did the same. When a transaction is accepted, its change is added to the set as "unconfirmed change" that only the sender can spend. Payments to other people appear only after mining, so a member still cannot spend a reward before it is confirmed. Figure 7 shows the library's change outputs being confirmed one after another.

5.2 Account-based model (account.c)

Each ledger ID (the member IDs, LIBRARY and the miner accounts) has an account with a balance, a nonce, a reserved amount, a pending count and a pointer to its history list. Every member from members.txt is created with a balance of 0 when the program starts.

- **Debit and credit:** confirming a transaction takes the amount plus the fee from the sender, gives the amount to the receiver, and adds 1 to the sender's nonce. The fee goes to the miner.
- **Nonce:** the nonce counts the sender's outgoing transactions. A new transaction must use exactly the next nonce (the confirmed nonce plus the transactions already waiting in the pool). A nonce that was already used is rejected as a replay. A nonce that skips ahead is rejected because it leaves a gap (Figure 14). For manual transfers the user types the nonce, so I can show the rejection. For reward transactions the program uses the next LIBRARY nonce by itself.
- **Not enough balance:** a transfer is rejected if the balance minus the reserved amount is less than the amount plus the fee. The reserved amount is the cost of the sender's transactions that are still in the pool. This stops the same coins from being promised twice.
- **History as a linked list:** each account has a linked list of TxHistoryNode items, with head and tail pointers so adding at the end is fast. Each node keeps the transaction ID, sender, receiver, amount, fee, nonce and time. Menu option 12 walks the list and prints the full

history of any account (Figure 16). The nodes are created before anything is changed, so if malloc fails nothing is left half done. All lists are freed when the program exits.

5.3 Choosing the model

The model is chosen with the option --model utxo or --model account. If the option is not given, a menu asks at startup. The choice is used for every transaction in the session. Inside ledger.c, each action (accept, release, confirm, balance) is sent to utxo.c or account.c, so the rest of the program does not need to know which model is used.

5.4 Comparison

Table 3: UTXO and account models compared

	UTXO model	**Account model**
Where the balance is	Not stored. It is the total of unspent outputs	Stored in each account
Stopping replays and double spends	Spent outputs are gone. Pending inputs are locked	The nonce must be exactly the next one. A used nonce is never accepted again
Fee	Inputs minus outputs	A field that is taken from the sender with the amount
Change	An extra output that goes back to the sender	Not needed. Only the balance changes
Order of transactions	Only decided by which outputs are spent	Strict order for each sender, by nonce
History	Built from the confirmed list	A linked list for each account
Good points	Outputs are independent, so they can be checked at the same time. Easy to audit	Simple to understand. Small state. Easy to show balances
Bad points	Choosing coins, many small outputs, handling change	Each transaction depends on the sender's current state. The nonce must be managed

In my tests, both models accepted and rejected the same transactions. The error messages show the different ideas. A replayed transfer is rejected as "input already spent" by the UTXO model and as "nonce already used" by the account model (Figures 12 and 15).

6. Mining Simulation Method

6.1 Proof of work

The block hash is SHA-256 of a header made from the index, previous_hash, the hash of the signed content (which includes token_reward and tx_id), the difficulty, mined_by, the signature

bytes and last of all the nonce. A block is valid when its hash, written in hexadecimal, starts with as many zeros as the difficulty. The difficulty can be 1 to 4 and the default is 2. It is set with --difficulty or with menu option 8. Any other number is rejected.

Only the nonce changes between attempts. So the miner hashes the fixed part of the header once and keeps a copy of that state. For each attempt it copies the state and hashes only the nonce digits. The result is the same as hashing the whole header, but it is faster. Each hexadecimal digit is 4 bits, so the chance that a hash meets difficulty d is 1 in 16^d. The expected number of attempts is 16^d. I checked this with the benchmark (menu option 9). It mines 20 sample blocks at each difficulty:

Table 4: Difficulty benchmark (20 blocks for each level)

Difficulty	Target	Expected attempts	Average measured	Min	Max	Average time
1	0...	16	15	2	68	0.02 ms
2	00...	256	250	9	701	0.16 ms
3	000...	4,096	4,497	116	15,265	1.97 ms
4	0000...	65,536	79,979	415	491,440	24.1 ms

Each extra zero makes the work 16 times bigger. A single block can take much more or much less than the average, because finding a good hash is random. In one cloud run, a difficulty 2 block needed 1,606 attempts instead of about 256 (Figure 24).

6.2 Solo mining

The logged-in user mines every pending block alone. For each block the program shows the nonce, the number of hash attempts, the time and the hash. At the end it shows the total and the average number of attempts next to the expected 16^d. The miner keeps the full reward: 50 coins for each block plus all the transaction fees. It is paid with a coinbase transaction to MINER_username.

6.3 Pool mining

The user picks 2 to 8 miners. Each miner gets a random hash rate from 100 to 500 attempts per round (the option --seed makes this repeatable). The miners work at the same time on different nonces. Miner k out of n tries k, k+n, k+2n and so on, so no hash is computed twice. In one round, each miner makes as many attempts as its hash rate, spread evenly over the round. The program handles all the attempts in time order and stops everyone at the first valid hash. So each miner's number of attempts follows its hash rate.

My first version ran each miner's turn one after the other and stopped a miner when it found its own good hash. At difficulty 2 several miners found one in the same round, so the attempts no longer matched the hash rates. The fastest miner got less than 10%. Handling the attempts in time order fixed this.

Rewards: the total reward (50 for each block plus fees) first loses the 2% pool fee, which goes to POOL_OPERATOR. The rest is shared with this formula: share = miner attempts divided by total attempts, times (total reward minus pool fee). It is worked out in whole cents. Any leftover cent from rounding goes to the operator, so every cent is counted. The program prints a table with the miner ID, hash rate, attempts, share percent, blocks found and reward (Figure 11).

6.4 Cloud mining

The user rents mining power for 1 to 5 rounds. The plan gives 400 attempts each round. It costs a rental fee of 20 coins each round, plus a maintenance fee of 10% of the round's mining reward. In each round the rented power mines pending blocks. If a block is not solved by the end of the round, the search carries on from the same nonce in the next round. If the pool is empty, the rented power does nothing, but the rental fee is still charged.

The reward of each round, minus the maintenance fee, is paid to the renter by a coinbase transaction. The maintenance fee goes to CLOUD_HOST. I assumed that the rental fee is paid to the provider outside the chain. After every round the program prints the total rewards, the total fees and the net amount so far. If the total fees are more than the total rewards, it prints a warning that the rental is unprofitable. At the end a table shows each round, then the gross earnings, the total fees paid and the net profit (Figures 17 and 18).

Whether a rental makes money depends on the difficulty. 400 attempts per round is about 1.6 blocks per round at difficulty 2, but only about 0.1 blocks at difficulty 3. Renting for more rounds than there are pending blocks also loses money.

7. Security: SHA-256, Signatures and Immutability

- **SHA-256 is used everywhere:** for block hashes, the hash of the signed content, transaction IDs and proof of work. The hash of every block depends on its token_reward and tx_id. Every tx_id depends on the sender, receiver, amount, fee and nonce of its transaction.
- **Digital signatures:** as in Formative 1, every block is signed with the ECDSA P-256 private key of the librarian who recorded it. The key file is encrypted with AES and is only opened at login. The signature is checked with the librarian's public key. The signature covers the reward and the transaction ID, so a miner or an attacker cannot change the reward without the librarian's key.
- **Full validation:** option 15, the startup check, and every action that adds or mines a block check four things for each block: (a) the stored hash equals the recalculated hash, (b) previous_hash matches the block before, (c) the signature is valid, and (d) the hash meets the block's difficulty. They also check that the block's tx_id exists in the ledger, that the transaction's hash is correct and that its amount matches the token reward. Every problem is listed, not just the first one.
- **Ledger check:** every transaction in ledger.dat is hashed again when it is loaded. If someone edits a transaction, its ID no longer matches and the program refuses to start (Figure 23).

- **Proof of work and immutability:** to change an old block, an attacker must redo its proof of work. The new hash of that block then breaks the previous_hash of the next block, so every later block would have to be mined again too. Figure 21 shows this attack. The late reward is changed from 5 to 10 and the block is mined again in 323 attempts. Validation still reports the invalid signature, the reward that no longer matches its transaction, and the broken link at the next block. While the chain is invalid, new blocks and mining are refused, so the change is never saved.
- **Access control:** only ADMIN users can change the difficulty, run the tamper demo or add users.

Limits: this is a simulation on one computer. An attacker who controls the folder and a librarian's password could rebuild the whole chain. A real network stays safe because many nodes would reject a chain with less total work. The mining payouts are protected by their transaction IDs, but no block points to them. Members also do not sign their own transfers, because the librarian records them. A real system would improve these points.

8. Screenshots

All screenshots are from real runs of the program. They were taken from a terminal session and passwords are hidden. I removed the repeated main menu from them to save space.

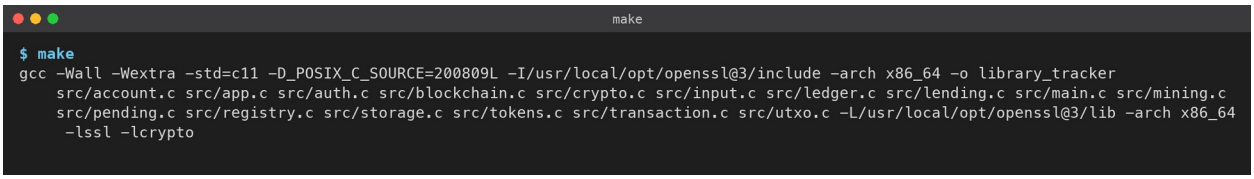


Figure 1: Compiling with make (no warnings with -Wall -Wextra)

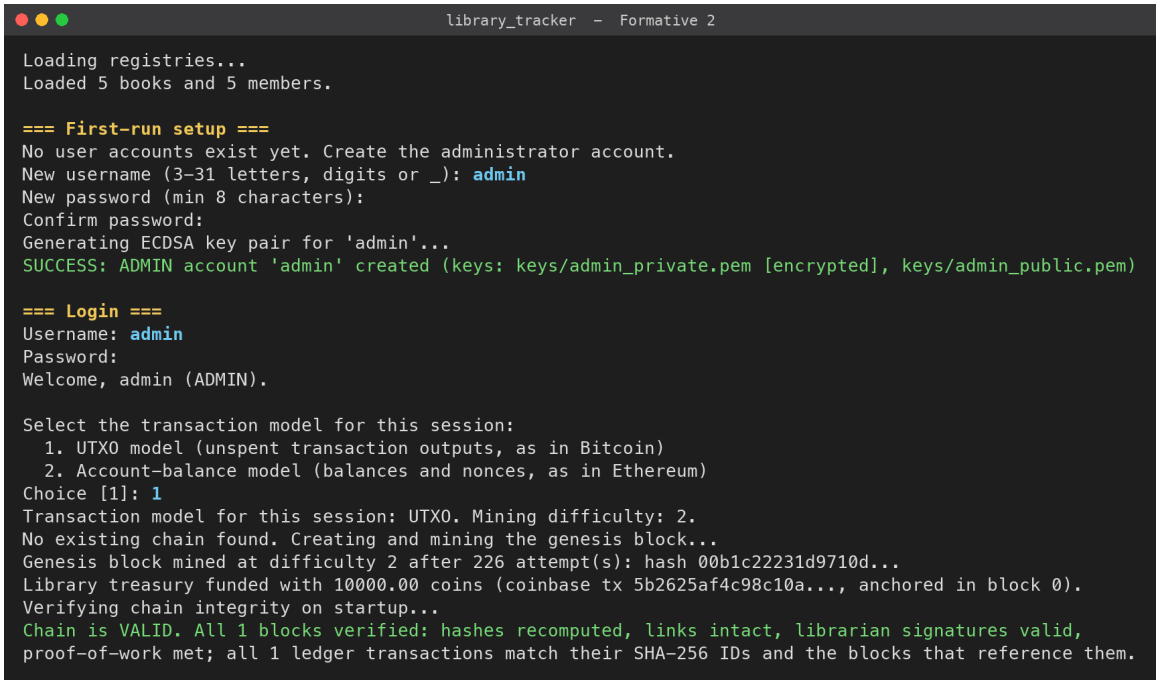


Figure 2: First run: admin account, model choice, genesis block mined and LIBRARY funded

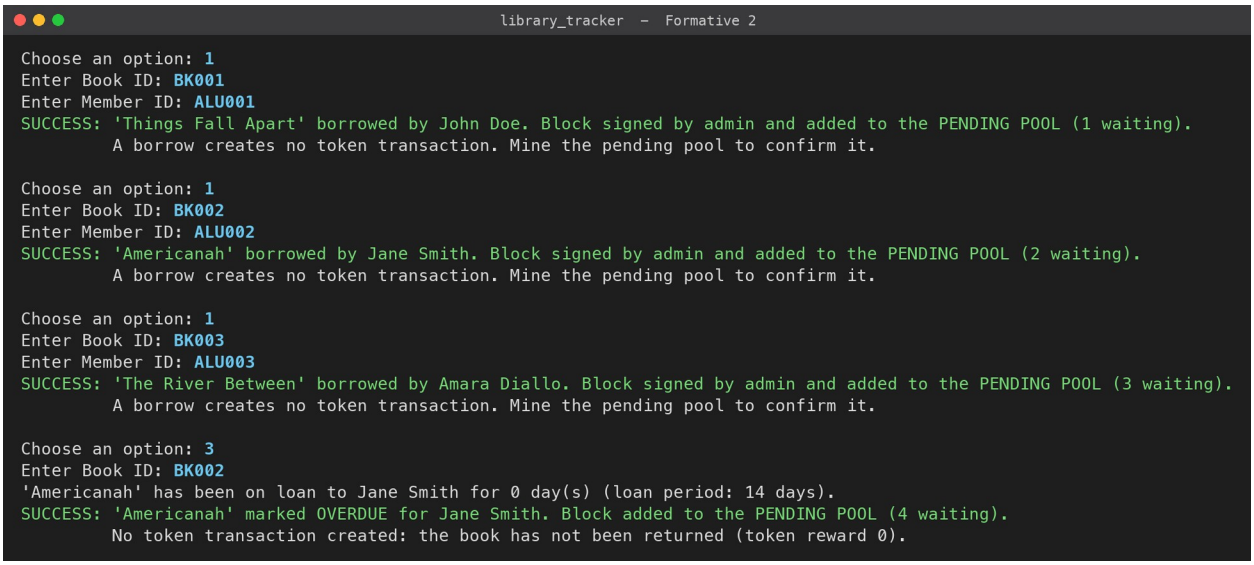


Figure 3: Borrows go to the pending pool. Marking a book overdue creates no transaction

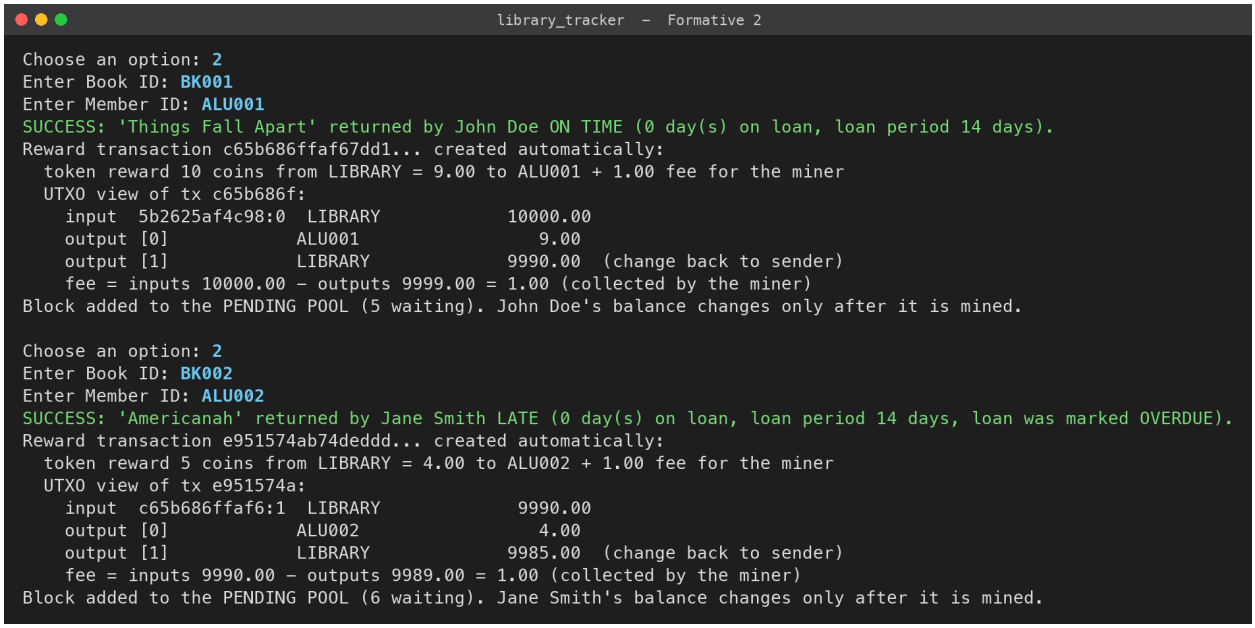


Figure 4: Returns create reward transactions by themselves: on time 10 coins, late 5 coins (UTXO view)

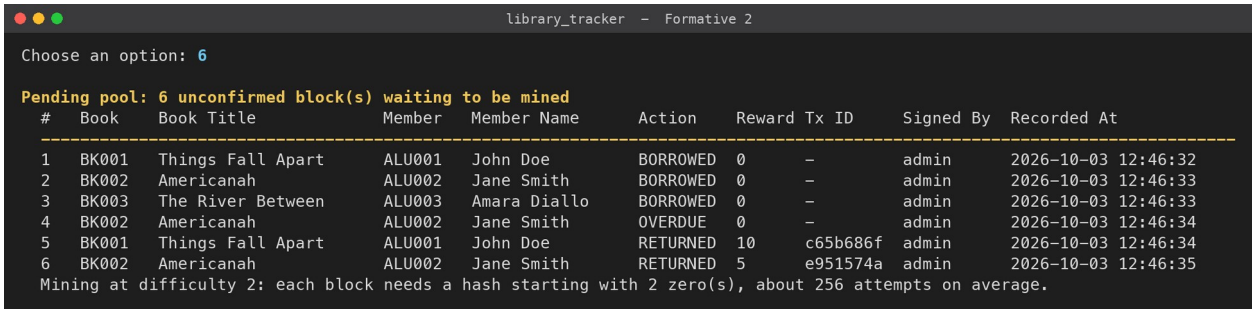


Figure 5: Pending pool with six blocks that are not mined yet

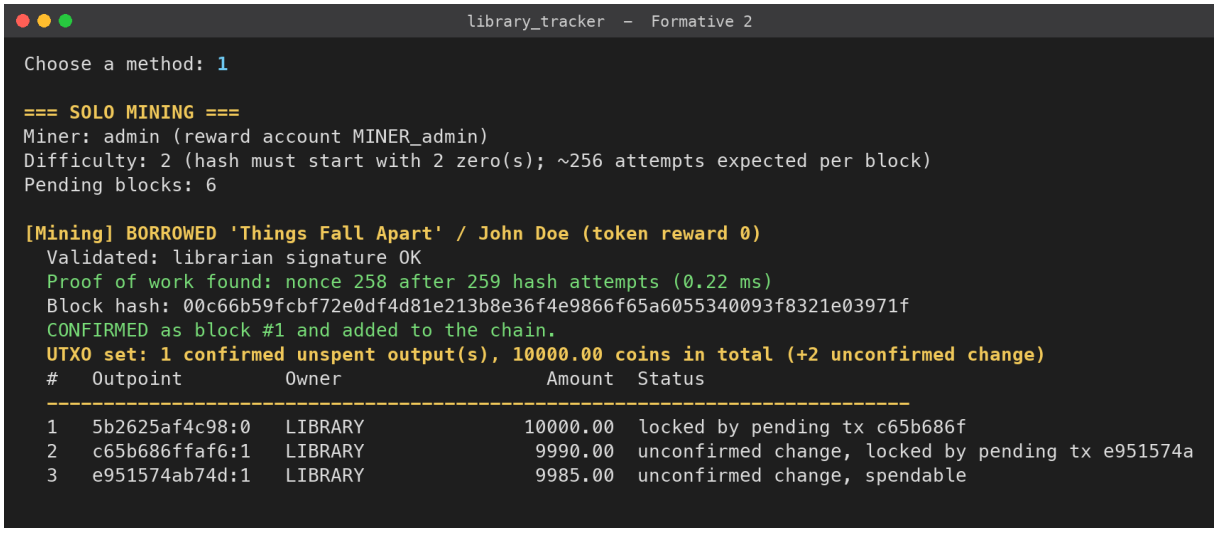


Figure 6: Solo mining: checks, nonce search, hash attempts and the UTXO set after the block

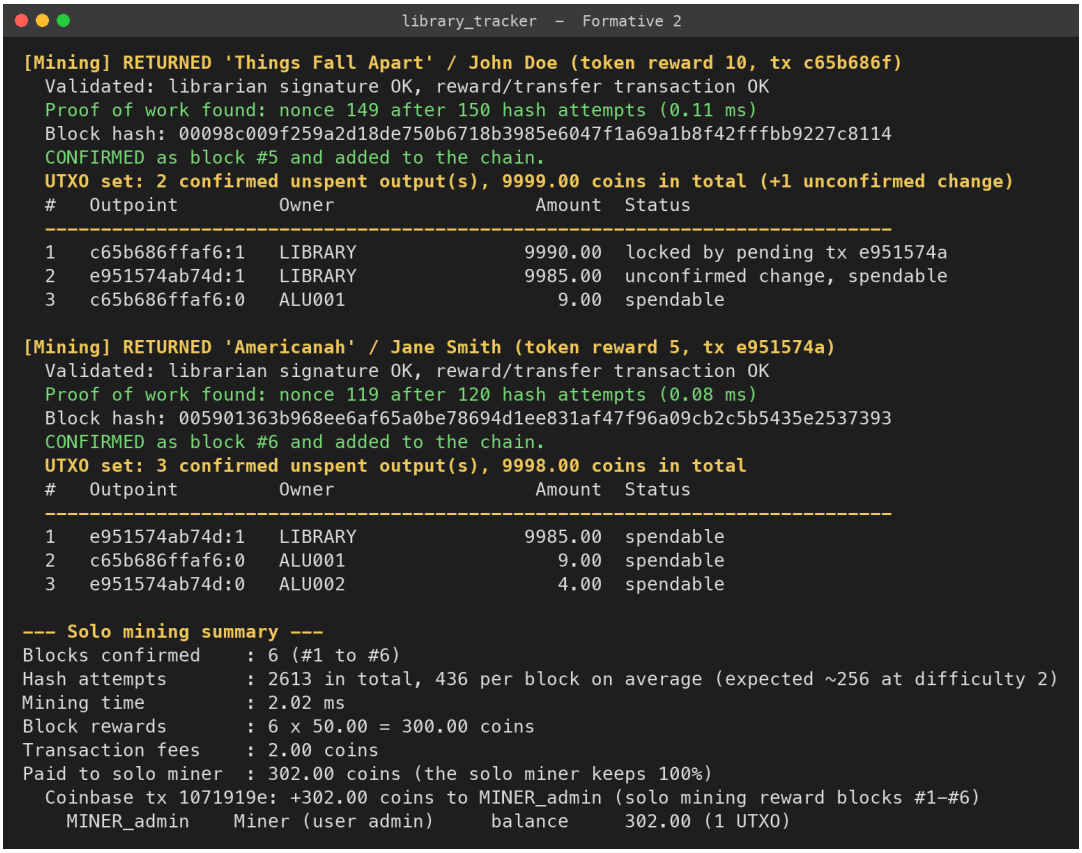


Figure 7: Blocks confirmed, members paid, and the solo mining summary with the miner's reward

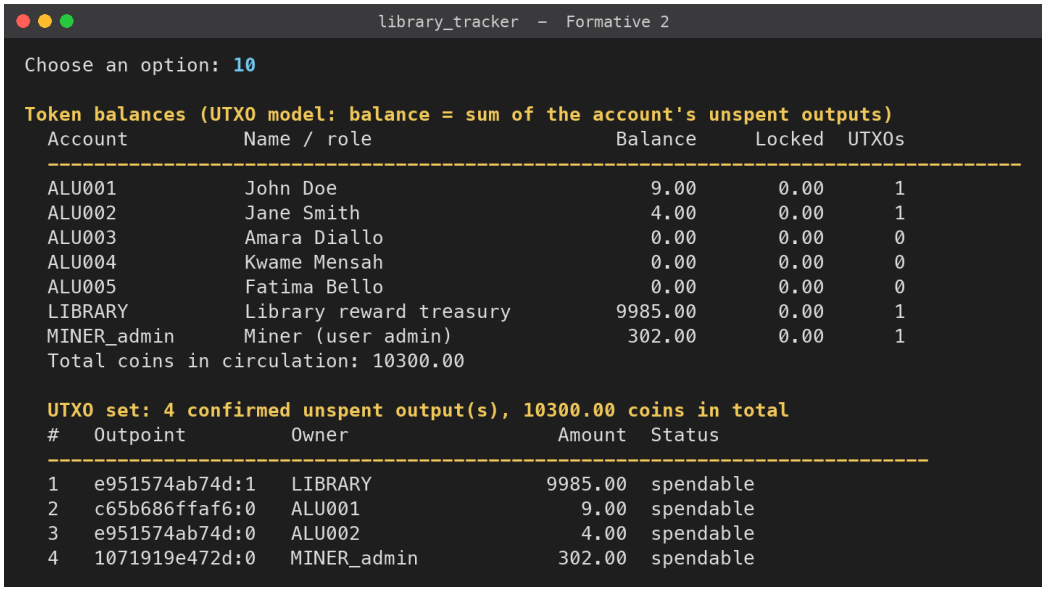


Figure 8: Balances and the full UTXO set after mining (UTXO model)

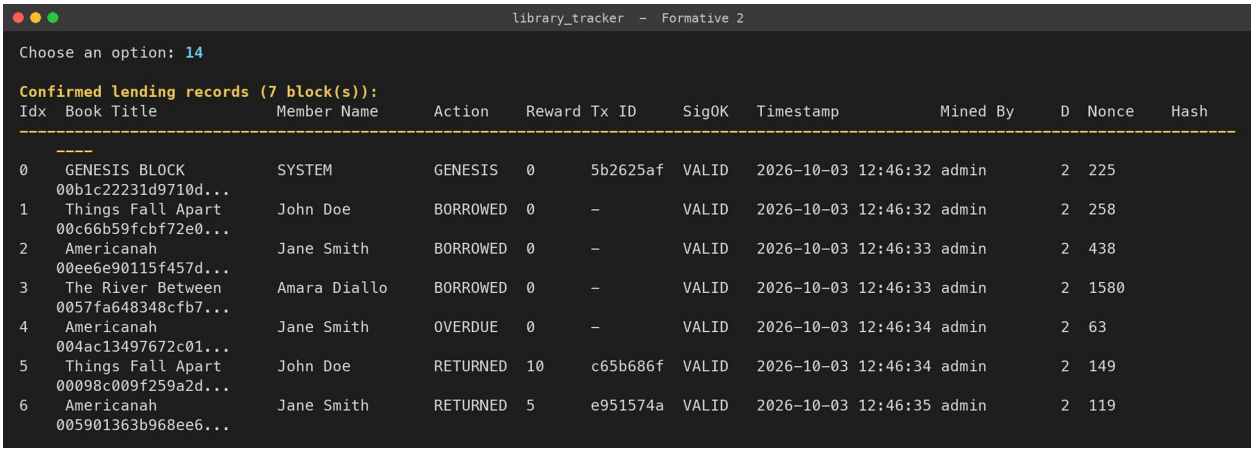


Figure 9: Confirmed lending records with token reward, transaction ID, signature check, miner, difficulty and nonce

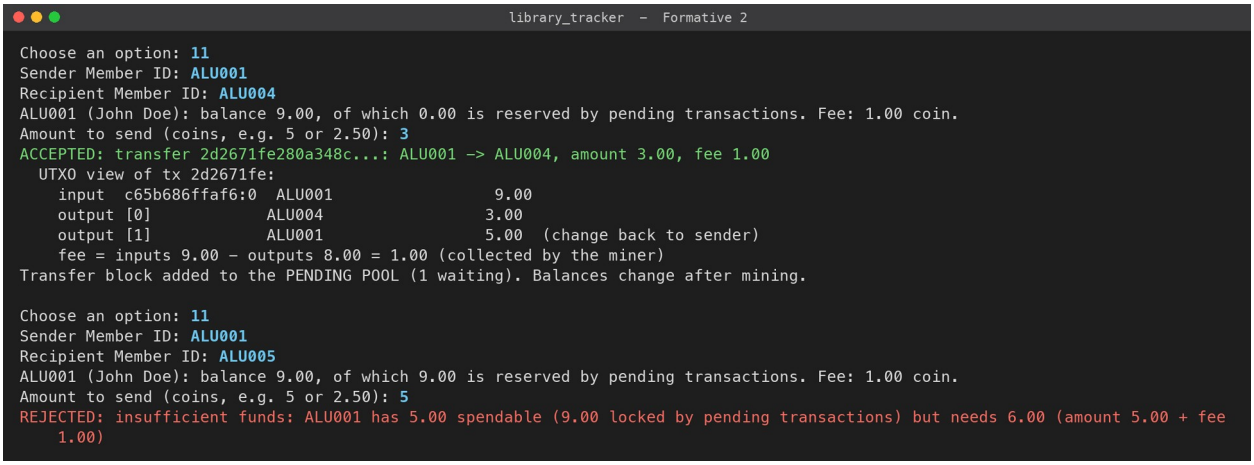


Figure 10: UTXO transfer with a change output, then a transfer rejected because the inputs are locked

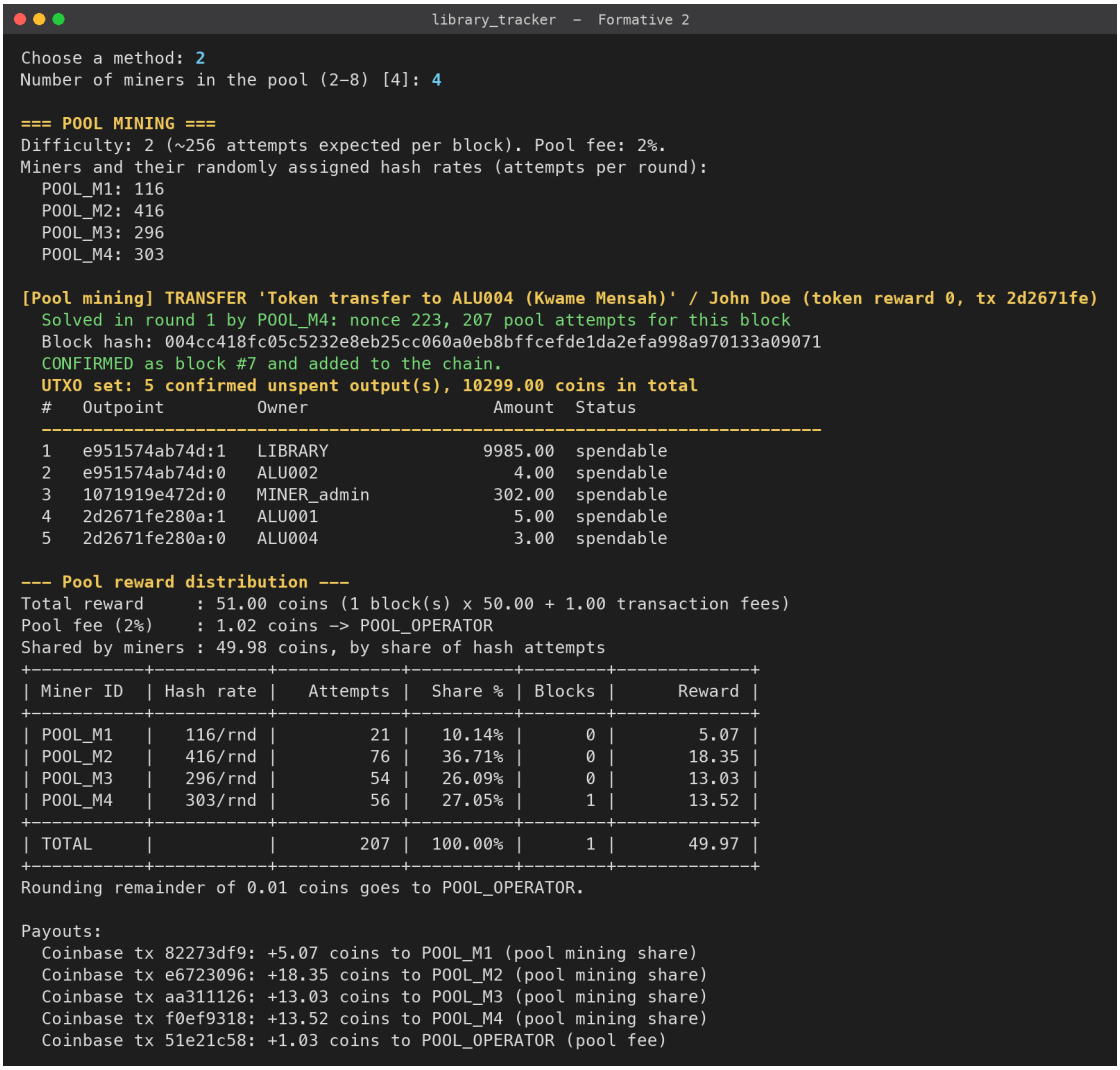


Figure 11: Pool mining: random hash rates, 2% pool fee and the reward table

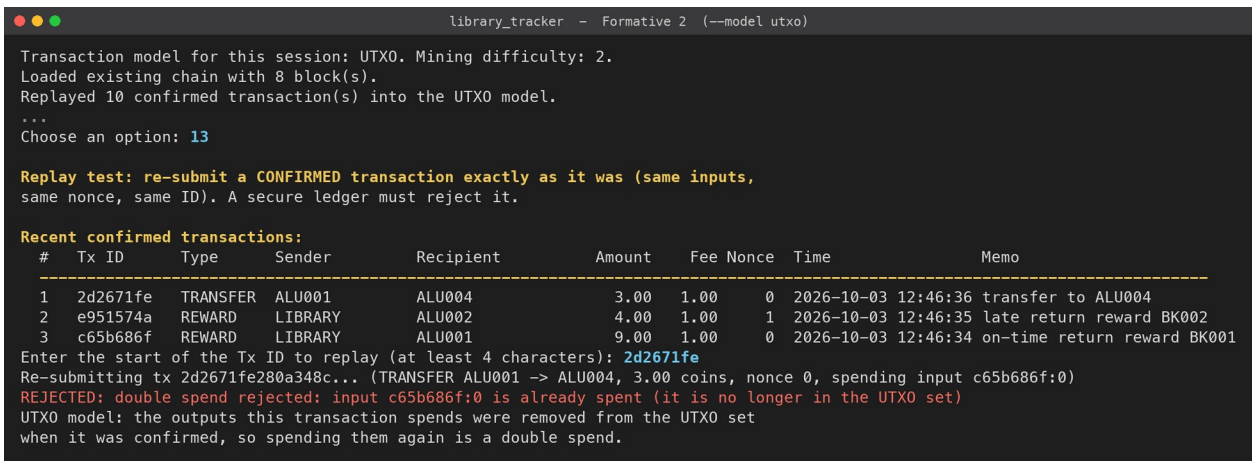


Figure 12: Double spend: sending a confirmed transfer again is rejected (UTXO model)

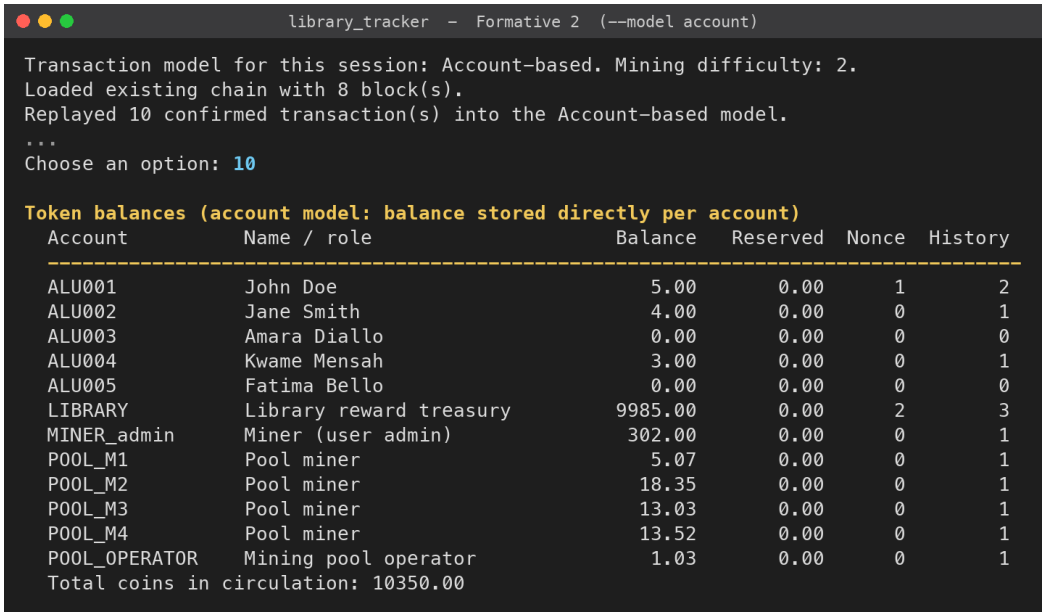


Figure 13: The same list replayed into the account model gives the same balances

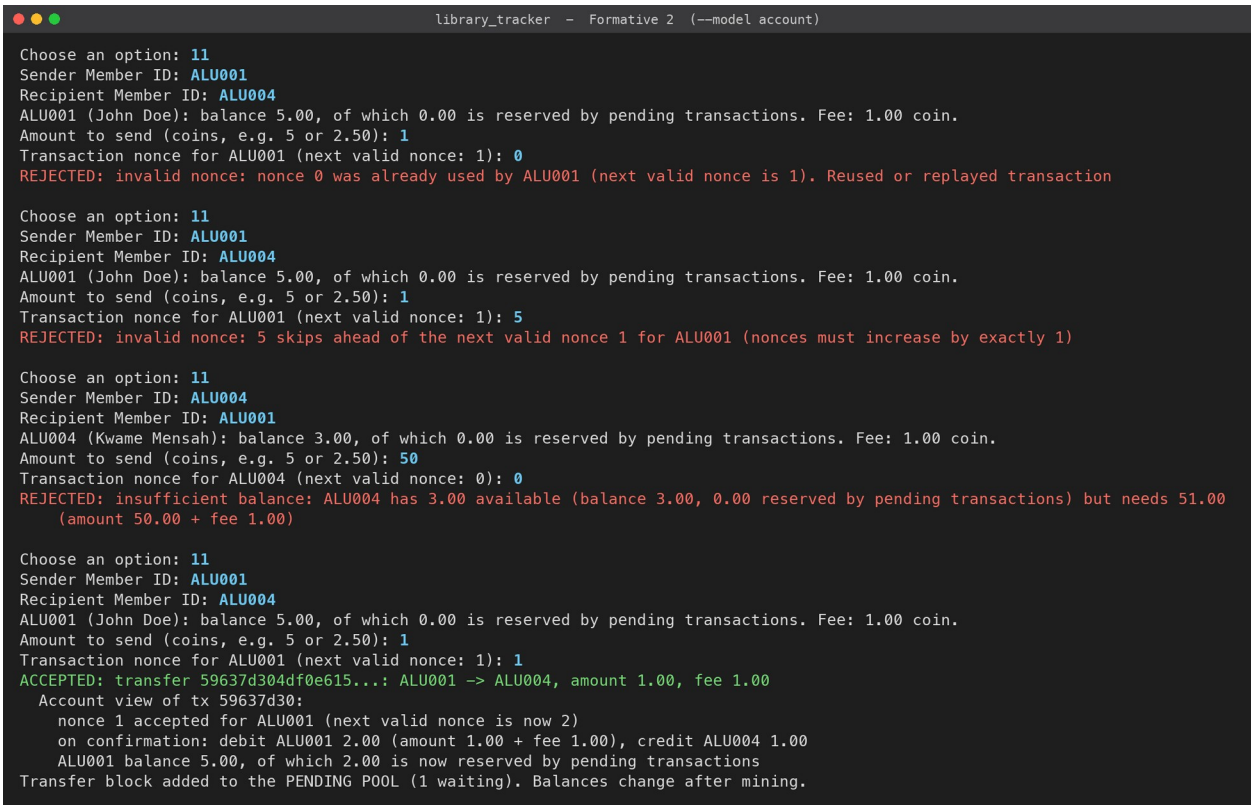


Figure 14: Account model: reused nonce, skipped nonce and not enough balance are rejected. The correct nonce is accepted

```
library_tracker  –  Formative 2  (--model account)

Choose an option: 13

Replay test: re-submit a CONFIRMED transaction exactly as it was (same inputs,
same nonce, same ID). A secure ledger must reject it.

Recent confirmed transactions:
  #   Tx ID      Type      Sender         Recipient          Amount    Fee Nonce  Time                Memo
  ------------------------------------------------------------------------------------------------------------------------
  1   2d2671fe   TRANSFER  ALU001         ALU004               3.00   1.00     0  2026-10-03 12:46:36 transfer to ALU004
  2   e951574a   REWARD    LIBRARY        ALU002               4.00   1.00     1  2026-10-03 12:46:35 late return reward BK002
  3   c65b686f   REWARD    LIBRARY        ALU001               9.00   1.00     0  2026-10-03 12:46:34 on-time return reward BK001
Enter the start of the Tx ID to replay (at least 4 characters): 2d2671fe
Re-submitting tx 2d2671fe280a348c... (TRANSFER ALU001 -> ALU004, 3.00 coins, nonce 0)
REJECTED: invalid nonce: nonce 0 was already used by ALU001 (next valid nonce is 2). Reused or replayed transaction
Account model: the sender's nonce moved past 0 when this transaction was
confirmed, so the same nonce can never be accepted again.
```

Figure 15: Replay rejected by the account model (nonce already used)

```
library_tracker  –  Formative 2  (--model account)

Choose an option: 12
Account ID (member ID, LIBRARY, MINER_<user>, POOL_M1, ...): ALU001

Transaction history for ALU001 (linked list, 3 entries, oldest first)
Balance: 3.00 | next nonce: 2 | pending outgoing: 0
  #   Tx ID      Type      Sender         Recipient          Amount    Fee Nonce  Time                Memo
  ------------------------------------------------------------------------------------------------------------------------
  1   c65b686f   REWARD    LIBRARY        ALU001               9.00   1.00     0  2026-10-03 12:46:34 on-time return reward BK001
  2   2d2671fe   TRANSFER  ALU001         ALU004               3.00   1.00     0  2026-10-03 12:46:36 transfer to ALU004
  3   59637d30   TRANSFER  ALU001         ALU004               1.00   1.00     1  2026-10-03 12:46:55 transfer to ALU004
```

Figure 16: Transaction history printed from the account's linked list

```
library_tracker  –  Formative 2  (--difficulty 1)

=== CLOUD MINING ===
Rental plan: 400 hash attempts per round, rental fee 20.00 coins per round,
maintenance fee 10% of each round's mining reward. Rewards go to MINER_admin.
At difficulty 1 a block needs ~16 attempts, i.e. ~25.00 block(s) per round.
Rental duration in rounds (1-5): 2

-- Round 1 of 2 --
  Working on: BORROWED 'Things Fall Apart' / Jane Smith (token reward 0)
  Proof of work found: nonce 16, hash 0f380e71beb8becadebf...
  CONFIRMED as block #11 and added to the chain.
...
  Pending pool is empty: the rented hash power is idle for the rest of this round.
  Round 1: 25 attempts, 3 block(s) confirmed, reward 152.00, rental fee 20.00, maintenance 15.20
  Coinbase tx c4691c0d: +136.80 coins to MINER_admin (cloud mining reward)
  Coinbase tx ad69058c: +15.20 coins to CLOUD_HOST (cloud maintenance fee)
  Cumulative: rewards 152.00, fees 35.20, net 116.80

-- Round 2 of 2 --
  Pending pool is empty: the rented hash power is idle for the rest of this round.
  Round 2: 0 attempts, 0 block(s) confirmed, reward 0.00, rental fee 20.00, maintenance 0.00
  Cumulative: rewards 152.00, fees 55.20, net 96.80

...
--- Cloud mining: multi-round earnings summary ---
Round  Attempts Blocks     Reward     Rental     Maint.   Cum.Reward   Cum.Fees    Cum.Net  Status
------------------------------------------------------------------------------------------------------------------------
1            25      3     152.00      20.00      15.20       152.00      35.20     116.80  ok
2             0      0       0.00      20.00       0.00       152.00      55.20      96.80  ok
------------------------------------------------------------------------------------------------------------------------
Gross earnings   : 152.00 coins
Total fees paid  : 55.20 coins (rental 40.00 + maintenance 15.20)
Net profit       : 96.80 coins
The rental was profitable in every round.
(Rental fees are billed by the provider off-chain; the ledger receives the reward minus maintenance.)
    MINER_admin    Miner (user admin)     balance     579.70 (4 UTXOs)
```

Figure 17: Cloud mining at difficulty 1: a profitable rental with the multi-round summary

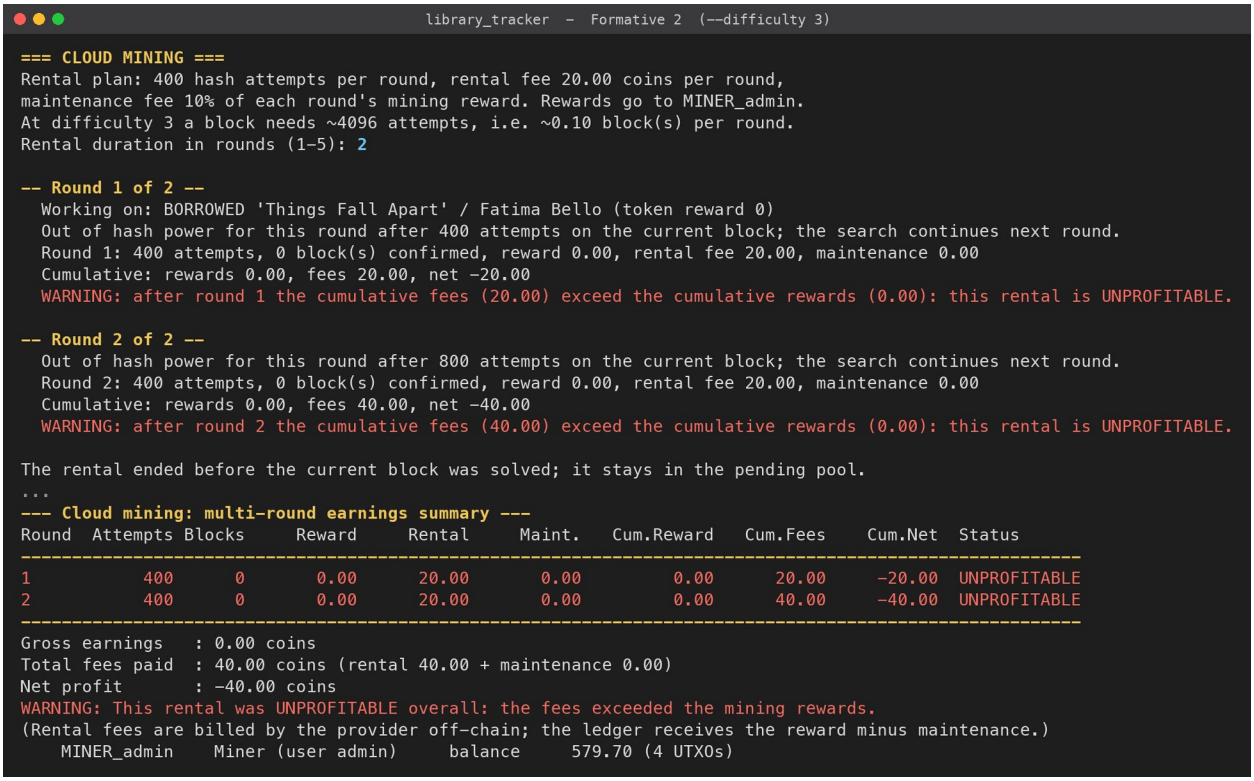


Figure 18: Cloud mining at difficulty 3: warnings that the rental is unprofitable

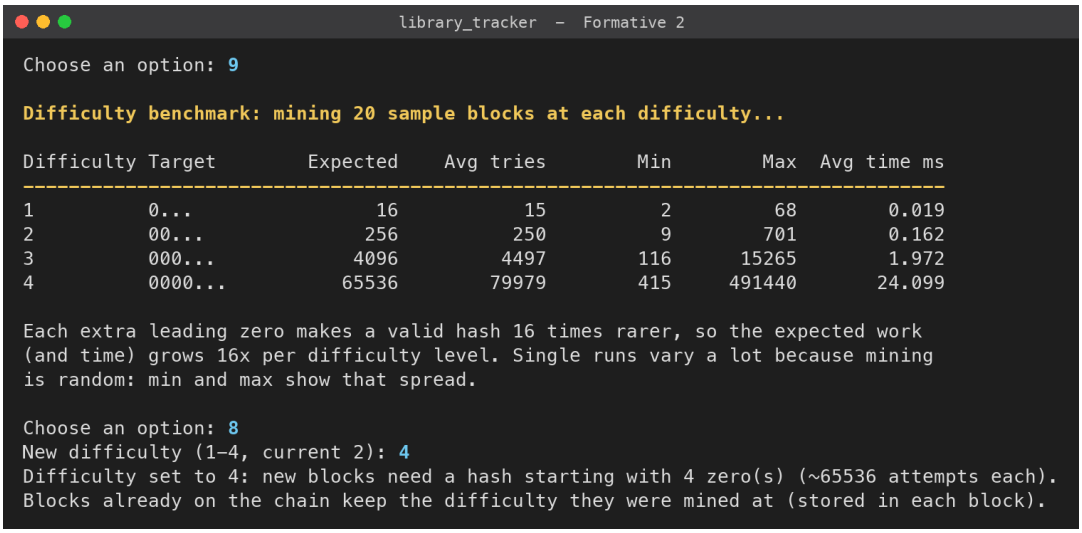


Figure 19: Difficulty benchmark, then the difficulty set to 4 from the menu

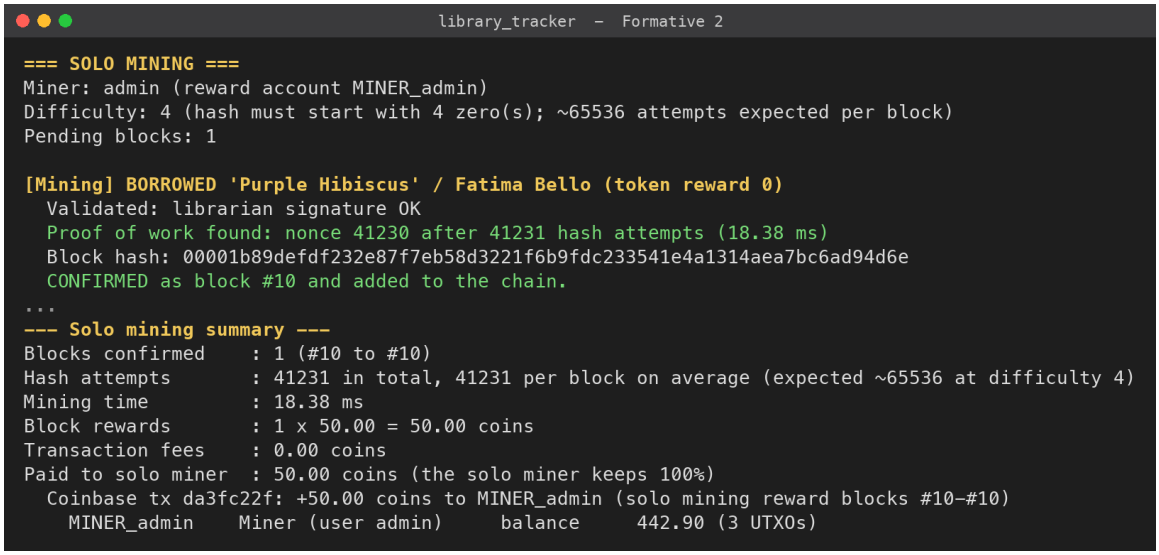


Figure 20: Solo mining at difficulty 4: 41,231 attempts, and the hash starts with 0000

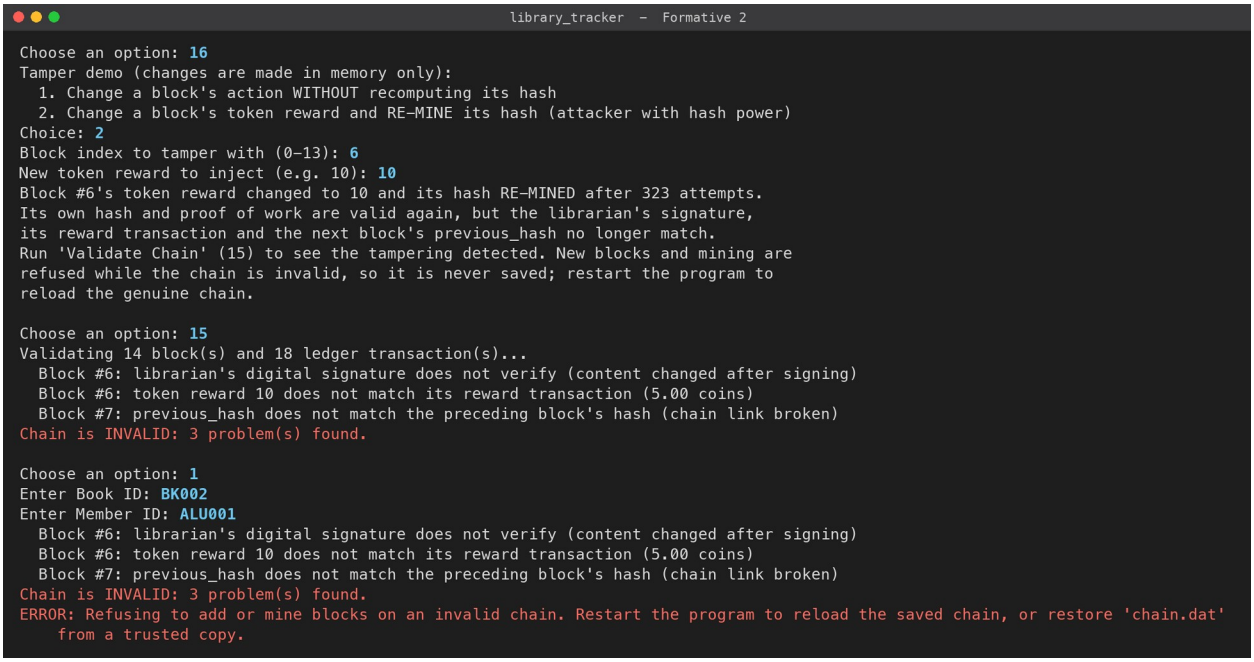


Figure 21: Attack: the reward is changed and the block is mined again. Validation still finds three problems and new blocks are refused

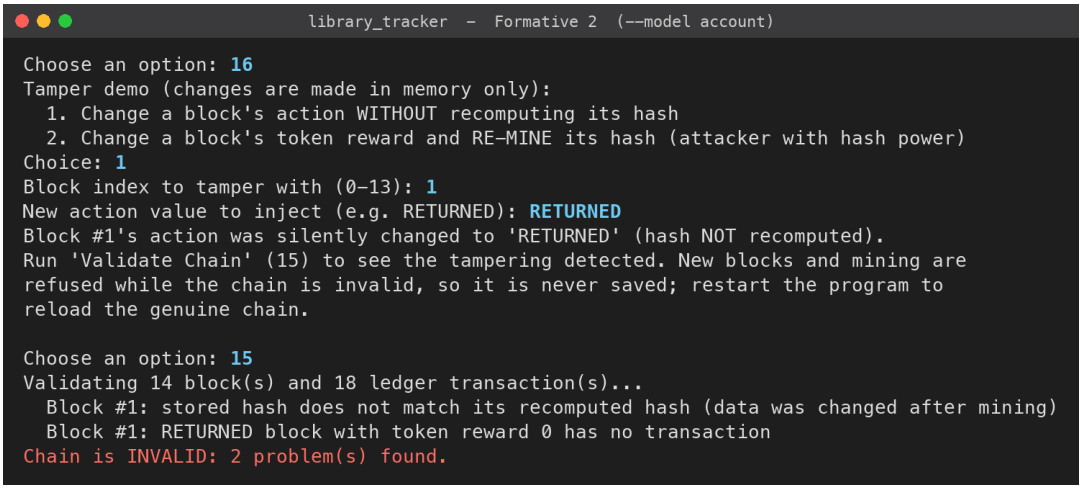


Figure 22: Changing an action without a new hash is detected

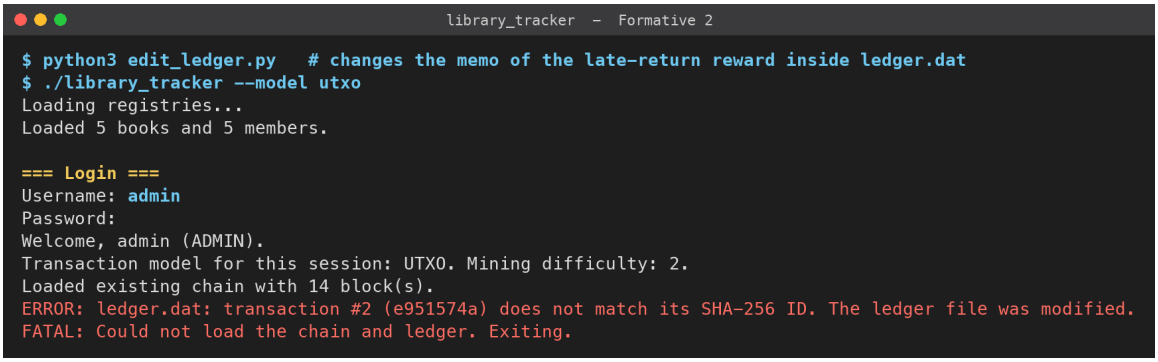


Figure 23: Editing a transaction in ledger.dat is detected at startup

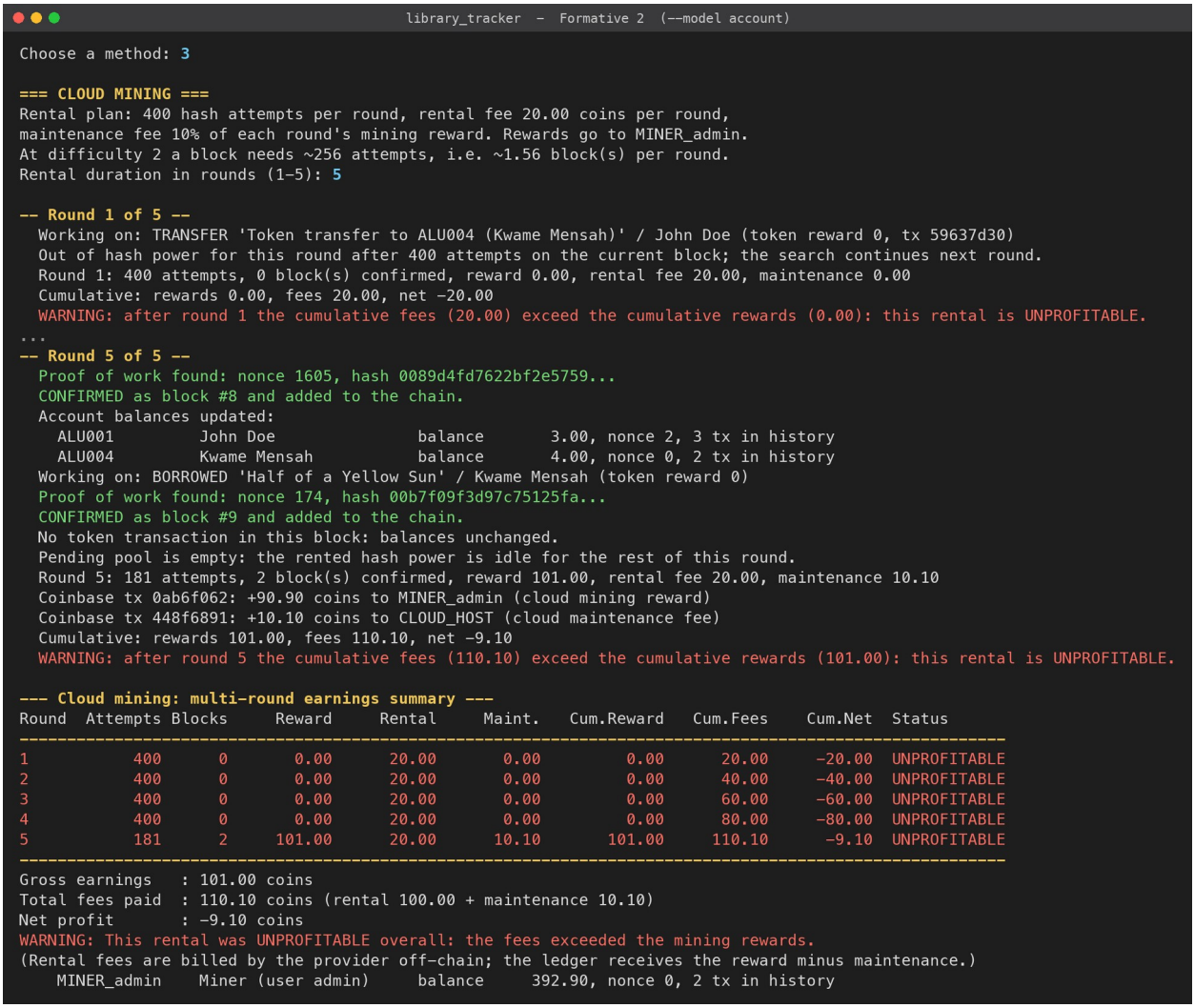


Figure 24: A run where an unlucky block needed 1,606 attempts at difficulty 2 (mining is random)

9. Test Results for Edge Cases

I tested the program in three ways:

- **Automated tests (make test):** the file tests/run_tests.sh runs the real program through every case in the brief, in a temporary folder. It types the menu input and checks the output. The 51 checks cover rewards, the pending pool, the three mining methods, both models, double spends, nonces, difficulty, tamper detection and access control. All 51 pass and it takes about 3 seconds.
- **Manual runs:** the cases in the screenshots, and also switching models between runs while some transactions were still pending.
- **Tools:** the program compiles with no warnings under -Wall -Wextra and passes the Clang static analyzer. It ran with no errors under AddressSanitizer and UndefinedBehaviorSanitizer. The macOS leaks tool reported "0 leaks for 0 total leaked bytes".

The table lists the edge cases and their results.

Table 5: Edge-case test results

#	Test case	Model	Expected	Result
1	Book marked OVERDUE and not returned	Both	Block with reward 0, no transaction, no balance change	Pass (Fig. 3, 9)
2	On-time return	Both	Reward 10: member gets 9, miner gets the 1 fee after mining	Pass (Fig. 4, 7)
3	Late return (loan marked overdue)	Both	Reward 5: member gets 4	Pass (Fig. 4, 7)
4	Balance before mining	Both	No change until the block is mined	Pass (Fig. 4, 7)
5	Transfer bigger than the balance	Account	Rejected: not enough balance	Pass (Fig. 14)
6	Transfer while the inputs are locked by a pending transfer	UTXO	Rejected: not enough funds (locked)	Pass (Fig. 10)
7	Spending a reward that is not mined yet	Both	Rejected: 0 spendable	Pass
8	Reused nonce (0 when 1 is expected)	Account	Rejected: nonce already used	Pass (Fig. 14)
9	Skipped nonce (5 when 1 is expected)	Account	Rejected: nonce skips ahead	Pass (Fig. 14)
10	Replay of a confirmed transfer	UTXO	Rejected: input already spent	Pass (Fig. 12)
11	Replay of a confirmed transfer	Account	Rejected: nonce already used	Pass (Fig. 15)
12	Change output	UTXO	Inputs 9 = 3 payment + 5 change + 1 fee	Pass (Fig. 10)
13	Cloud rental of 5 rounds for only 2 pending blocks	Account	Warning when fees are more than rewards	Pass (Fig. 24)
14	Cloud rental at difficulty 3	UTXO	Unprofitable warning each round, block stays pending	Pass (Fig. 18)
15	--difficulty 7 or menu value 5	-	Rejected (range is 1 to 4)	Pass
16	Pool shares	UTXO	Shares follow hash rate, 2% fee, total adds up exactly	Pass (Fig. 11)
17	Switching model between runs	Both	Same balances and total coins	Pass (Fig. 8, 13)

18	Pending reward made in account mode, mined in UTXO mode	Both	Accepted again and confirmed	Pass
19	Reward changed and block mined again	Both	Signature, transaction and link errors	Pass (Fig. 21)
20	chain.dat or ledger.dat edited on disk	Both	Found at startup	Pass (Fig. 23)
21	Librarian tries to change the difficulty or tamper	Both	Access denied	Pass
22	Chain file from Formative 1	-	Refused, asks for make clean	Pass

10. Design Choices and Assumptions

- **One block for each lending event.** I kept the Formative 1 design. The brief asks to show each confirmed record with its own title, member, action, time, reward and signature. One mining run confirms the whole pending pool, one block at a time.
- **A treasury pays the rewards.** Rewards are real transfers from a LIBRARY account that is funded in the genesis block. They are not new coins. So reward transactions have inputs, change and nonces like any other transaction.
- **Fee.** The brief says the fee is taken from the token amount before the member is paid. So token_reward stays 10 or 5 in the block, the member gets the reward minus 1, and the fee goes to the miner.
- **Mining rewards.** 50 coins for each block plus fees. They are paid by coinbase transactions to MINER_username, to the POOL_M accounts and POOL_OPERATOR, or to the renter and CLOUD_HOST.
- **One list for both models.** The transaction ID only hashes the fields that both models share. The UTXO model works out its own inputs and outputs. So one ledger.dat can be replayed into either model.
- **Whole-number amounts.** All amounts are kept in hundredths of a coin, so money is never stored as a decimal number.
- **Safe saving.** chain.dat, pending.dat and ledger.dat start with a label and the record size. They are written to a temporary file first and then renamed, so a crash cannot leave a half-written file.
- **Assumptions.** The loan period is 14 days. Members have no keys, so the logged-in librarian records and signs transfers for them. The cloud rental fee is paid outside the chain. The pool miners are simulated inside one program.

11. Problems and Solutions

- **Signing before the place in the chain is known.** In Formative 1 the signature covered the index and previous_hash. A pending block does not have these yet. So I split the block into signed content (the lending event, the reward and tx_id) and a header that the miner fills in. The block hash covers both.
- **A second return was refused in the UTXO model.** As explained in section 5.1, the library's single output was locked by the first pending reward. I let a sender spend its own unconfirmed change.
- **Unfair pool shares.** As explained in section 6.3, I replaced the one-miner-at-a-time loop with attempts handled in time order.
- **Switching models with transactions still pending.** Pending transactions are accepted again under the new model when the program starts. The UTXO-only data is cleared when a transaction is confirmed under the account model. So the list can always be replayed under either model.
- **Mining speed.** Hashing the whole header for every attempt was slow, so I hash the fixed part once and copy that state for each nonce. Difficulty 4 (about 65,000 attempts) takes about 20 to 40 ms.

12. Conclusion

The new version joins lending, token transactions and mining in one flow. A return creates a reward transaction that is linked to its signed block. The block waits in the pending pool until a miner confirms it with SHA-256 proof of work. Only then does the member's balance change. The same transactions can be handled with the UTXO model (inputs, outputs, change, fees and locked inputs) or the account model (balances, nonces and a linked-list history). Solo, pool and cloud mining all confirm the same pending pool and pay their rewards. Validation finds any change to a block, a transaction or the chain links, even if the attacker does the proof of work again.

Build and run steps are in README.md in the repository:
https://github.com/goldensash1/Assignment-2--F-A-Extending-the-Blockchain-Based-Library-Book-Lending-Tracker

Demo video: https://youtu.be/A9BjZngE3Ks